

TRƯỜNG ĐẠI HỌC CÔNG NGHIỆP HÀ NỘI  
KHOA CÔNG NGHỆ THÔNG TIN

ĐỒ ÁN TỐT NGHIỆP

XÂY DỰNG NỀN TẢNG ENGLISHAITUTOR  
GIA SƯ AI, QUIZ CÁ NHÂN HÓA VÀ LỘ TRÌNH HỌC  
KHÉP KÍN

| Thông tin            | Nội dung                                                         |
|----------------------|------------------------------------------------------------------|
| Sinh viên thực hiện  | Nguyễn Việt Anh                                                  |
| Mã sinh viên         | 2021600938                                                       |
| Lớp                  | HTTT01-K16                                                       |
| Giảng viên hướng dẫn | PGS.TS. Nguyễn Thị Kim Sơn                                       |
| Phiên bản báo cáo    | Cập nhật toàn bộ chức năng MVP sau giai đoạn hoàn thiện sản phẩm |

Hà Nội - 2026

## LỜI CẢM ƠN

Em xin gửi lời cảm ơn chân thành tới các thầy cô trong Khoa Công nghệ thông tin đã hướng dẫn, góp ý và tạo điều kiện để em hoàn thành đồ án tốt nghiệp. Trong quá trình thực hiện EnglishAltutor, em nhận được nhiều phản hồi thực tế về trải nghiệm luyện nói, chất lượng quiz, cách trình bày giao diện và khả năng vận hành sản phẩm.

Những góp ý đó giúp em điều chỉnh định hướng từ một khung chat AI đơn giản sang một nền tảng học tập có luồng rõ ràng: gia sư AI, bài tập trong chat, kho quiz, review lỗi, thi nâng cấp, quản trị nội dung và gói học. Báo cáo này được viết lại theo hướng sản phẩm hoàn chỉnh hơn để trình bày được giá trị khác biệt của hệ thống.

Do thời gian và kinh nghiệm còn hạn chế, đồ án vẫn còn các hướng có thể tiếp tục cải tiến như tích hợp thanh toán tự động, mở rộng ngân hàng đề có kiểm định và đánh giá phát âm sâu hơn. Em mong tiếp tục nhận được ý kiến đóng góp của thầy cô để hệ thống hoàn thiện hơn.

## TÓM TẮT ĐỒ ÁN

EnglishAltutor là nền tảng học tiếng Anh trên web, tập trung vào việc biến mỗi phiên luyện tập thành dữ liệu học tiếp theo. Người học không chỉ chat với AI mà còn có thể luyện nói bằng giọng nói, nhận transcript, nghe phản hồi TTS, làm quiz trắc nghiệm ngay trong khung chat, xem lại lỗi sau phiên, làm quiz ôn lỗi cá nhân và thực hiện bài thi nâng cấp trình độ.

Hệ thống gồm frontend React/TypeScript/Vite/TailwindCSS, backend FastAPI, agent worker dùng LiveKit Agents, LangGraph để điều phối hội thoại, PostgreSQL để lưu user, phiên học, lỗi, quiz, kết quả, refresh token và payment request. LiveKit được dùng cho WebRTC, audio track, text stream và widget stream. Agent tích hợp STT Whisper, TTS EdgeTTS, VAD Silero và LLM Gemini.

Điểm khác biệt cốt lõi so với ChatGPT thông thường là EnglishAltutor có vòng học khép kín: đăng nhập và hồ sơ CEFR, phiên học realtime, lưu lỗi, sinh review, tạo bài ôn lỗi, làm quiz theo bộ, AI review kết quả, thi nâng cấp và quản trị nội dung bởi admin. Điều này giúp sản phẩm có thể demo như một hệ thống học tập thực tế thay vì một chatbot đơn lẻ.

Từ khóa: EnglishAltutor, AI tutor, WebRTC, LiveKit, LangGraph, STT, TTS, CEFR, quiz, remedial learning, FastAPI, React, PostgreSQL.

## MỤC LỤC RÚT GỌN

1. Mở đầu: lý do chọn đề tài, mục tiêu, phạm vi và định hướng sản phẩm.
2. Chương 1: cơ sở lý thuyết về CEFR, AI tutor, WebRTC, STT/TTS, LangGraph và quiz remediation.
3. Chương 2: phân tích yêu cầu, tác nhân, use case, kiến trúc, dữ liệu và phân quyền.
4. Chương 3: thiết kế giao diện và luồng trải nghiệm learner/admin.
5. Chương 4: xây dựng hệ thống và các module code quan trọng.
6. Chương 5: kiểm thử, đánh giá MVP và kế hoạch demo 30 phút.
7. Kết luận, tài liệu tham khảo và phụ lục hỏi đáp bảo vệ.

# MỞ ĐẦU

## 1. Lý do chọn đề tài

Người học tiếng Anh thường thiếu môi trường luyện tập đều đặn, thiếu phản hồi ngay sau khi mắc lỗi và thiếu một lộ trình biến lỗi thành bài tập cụ thể. Nếu chỉ dùng một chatbot tổng quát, người học có thể hỏi đáp linh hoạt nhưng dữ liệu học tập dễ bị rời rạc, không có quản trị nội dung, không có quota/gói học và không có báo cáo tiến bộ.

EnglishAltutor được xây dựng để giải quyết khoảng trống đó. Hệ thống đặt gia sư AI vào một sản phẩm học tập có tài khoản, phân quyền, CEFR, kho quiz, review kết quả, dashboard và admin quản trị. Vì vậy đề tài có ý nghĩa thực tế hơn một demo hội thoại AI đơn thuần.

## 2. Mục tiêu đề tài

- Xây dựng web app có thể sử dụng thực tế trên trình duyệt, hỗ trợ đăng ký, đăng nhập, quên mật khẩu và phân quyền learner/admin.
- Tạo phiên Gia sư AI realtime qua LiveKit, hỗ trợ voice, text, transcript, TTS và STT.
- Cá nhân hóa phản hồi theo CEFR, topic, memory và lỗi trong phiên.
- Tạo quiz trắc nghiệm trong chat khi learner yêu cầu làm bài tập hoặc khi agent đề nghị sau một số lượt nói.
- Xây dựng kho quiz theo bộ, gồm trắc nghiệm, điền từ, nghe và nói; có AI review kết quả.
- Cho admin tạo/sửa/xóa/import quiz, quản lý user, đổi level và duyệt thanh toán QR.
- Thiết kế gói Free/Plus/Ultra với quota chat/quiz và popup/paywall khi hết lượt.

## 3. Phạm vi và định hướng MVP

Phiên bản MVP hiện tại bao phủ toàn bộ hành trình từ landing page, auth, học với gia sư AI, quiz, review, billing đến admin. Hệ thống chưa khẳng định thay thế chứng chỉ quốc tế; bài thi nâng cấp chỉ là đánh giá nội bộ trong sản phẩm. Thanh toán QR là bán tự động: người dùng tạo yêu cầu, chuyển khoản và admin duyệt.

### Điểm cần nhấn mạnh khi bảo vệ

- Sản phẩm không dừng ở chat: điểm chính là vòng học khép kín từ hội thoại → lỗi → quiz → review → lộ trình.
- AI được đặt trong kiến trúc có kiểm soát bằng prompt, LangGraph, schema output, database và role/quota.
- Các widget quiz/listening/speaking làm cho khung chat trở thành không gian học tương tác chứ không phải chỉ là text chat.

# CHƯƠNG 1. CƠ SỞ LÝ THUYẾT

## 1.1. Học tiếng Anh theo vòng phản hồi

Học ngoại ngữ hiệu quả cần vòng phản hồi liên tục: người học thực hành, nhận ra lỗi, sửa lỗi, luyện lại bằng bài tập và theo dõi tiến độ. Nếu một hệ thống chỉ trả lời hội thoại mà không lưu vết lỗi hoặc không tạo bài luyện tiếp theo, giá trị học tập sẽ khó chứng minh.

### 1.2. CEFR trong hệ thống học tập

CEFR chia trình độ thành A1, A2, B1, B2, C1 và C2. Trong EnglishAltutor, CEFR được dùng ở bốn điểm: hồ sơ người học, prompt của agent, độ khó quiz và quyền truy cập bài tập. Learner không tự ý đổi level; admin điều chỉnh để tránh người học chọn bài quá lệch trình độ.

### 1.3. AI tutor và prompt engineering

LLM có khả năng sinh hội thoại tự nhiên, phân tích lỗi và giải thích. Tuy nhiên để phù hợp giáo dục, hệ thống phải giới hạn hành vi bằng system prompt: dùng tiếng Anh, điều chỉnh theo CEFR, sửa lỗi bằng kỹ thuật recast, hỏi tiếp để duy trì hội thoại và không biến thành trợ lý tổng quát.

### 1.4. WebRTC, LiveKit, STT, TTS và VAD

WebRTC cho phép truyền audio realtime trên trình duyệt. LiveKit cung cấp room, token, participant, audio track, transcription event và text stream. STT chuyển lời nói thành văn bản; TTS tạo tiếng nói của tutor; VAD phát hiện điểm bắt đầu/kết thúc lượt nói để tránh agent chen ngang khi người học chưa nói xong.

### 1.5. LangGraph và điều phối hội thoại

LangGraph giúp chia logic agent thành các node có trạng thái, thay vì một prompt duy nhất. Với EnglishAltutor, node assess phân tích lỗi và CEFR gợi ý; node respond/correct tạo phản hồi; node save memory lưu thông tin dài hạn. Thiết kế này làm hành vi agent dễ giải thích khi bảo vệ.

### 1.6. Quiz remediation và đánh giá nội bộ

Quiz remediation là cách biến lỗi thật thành bài luyện lại. Ngoài quiz theo bộ, hệ thống còn tạo quiz ôn lỗi từ error\_logs và kết quả quiz sai. Bài thi nâng cấp kết hợp text, listening và speaking nhằm đánh giá nội bộ mức sẵn sàng chuyển CEFR, không thay thế IELTS/TOEIC/Cambridge.

| Nhóm công nghệ | Công nghệ                            | Vai trò trong hệ thống                                                     |
|----------------|--------------------------------------|----------------------------------------------------------------------------|
| Frontend       | React, TypeScript, Vite, TailwindCSS | Xây dựng SPA learner/admin, UI quiz, billing và practice room.             |
| Realtime       | LiveKit, WebRTC, TURN                | Kết nối audio, text stream, transcription và widget stream.                |
| AI Agent       | LangGraph, LiveKit Agents, Gemini    | Điều phối hội thoại, phân tích lỗi, tạo phản hồi, tạo quiz/review.         |
| Voice          | WhisperSTT, EdgeTTS, Silero VAD      | Nhận giọng nói, phát giọng tutor và phát hiện lượt nói.                    |
| Backend        | FastAPI, SQLAlchemy, Pydantic        | Auth, token, quiz, billing, admin, sessions.                               |
| Database       | PostgreSQL, pgvector, mem0           | Lưu user, phiên học, lỗi, quiz, kết quả, payment, refresh token và memory. |

## CHƯƠNG 2. KHẢO SÁT, PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

### 2.1. Bài toán sản phẩm

Bài toán không chỉ là tạo một chatbot tiếng Anh, mà là tạo nền tảng học tập có thể vận hành: người học có tài khoản, có level, có giới hạn gói học, có quiz thật để làm, có review điểm mạnh/yếu và có admin quản trị nội dung.

## 2.2. Điểm khác biệt so với ChatGPT thông thường

| Tiêu chí       | ChatGPT thông thường                             | EnglishAltutor                                                                |
|----------------|--------------------------------------------------|-------------------------------------------------------------------------------|
| Ngữ cảnh học   | Phụ thuộc người dùng tự nhắc lại                 | Lưu session, lỗi, quiz attempts và memory.                                    |
| Voice realtime | Có thể voice nhưng không gắn vào luồng học riêng | LiveKit room, transcript, STT/TTS, widget stream trong cùng phiên.            |
| Bài tập        | Người dùng tự yêu cầu                            | Quiz theo bộ, quiz trong chat, quiz ôn lỗi, listening/speaking, thi nâng cấp. |
| Quản trị       | Không có admin vận hành nội dung                 | Admin quản lý user, CEFR, quiz, payment request.                              |
| Sản phẩm       | Hỏi đáp tổng quát                                | Có landing, billing, quota, dashboard, review và luồng học khép kín.          |

## 2.3. Tác nhân

| Tác nhân      | Mục tiêu                    | Chức năng chính                                                        |
|---------------|-----------------------------|------------------------------------------------------------------------|
| Learner       | Học tiếng Anh theo trình độ | Gia sư AI, quiz, review, dashboard, billing, thi nâng cấp.             |
| Admin         | Vận hành hệ thống           | Quản lý user, đổi CEFR/gói, CRUD quiz, import nguồn, duyệt thanh toán. |
| AI Agent      | Gia sư và bộ tạo nội dung   | Phân tích lỗi, phản hồi, tạo widget, sinh quiz, review kết quả.        |
| Dịch vụ ngoài | Realtime/AI/email           | LiveKit, Gemini, EdgeTTS, SMTP/Gmail, QR generator.                    |

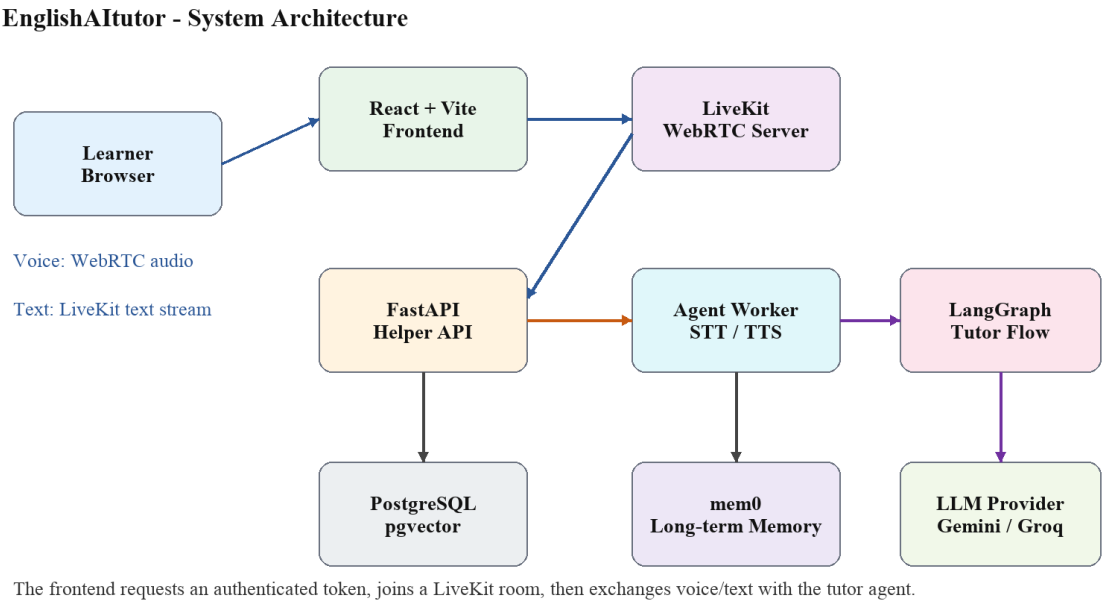
## 2.4. Nhóm chức năng

| Mã  | Nhóm chức năng     | Mô tả                                                                                |
|-----|--------------------|--------------------------------------------------------------------------------------|
| F01 | Xác thực và hồ sơ  | Đăng ký, đăng nhập, refresh token, logout, quên mật khẩu qua mã email, hồ sơ CEFR.   |
| F02 | Gia sư AI realtime | Bắt đầu phiên, voice/text, transcript, TTS, STT, VAD, kết thúc và lưu session.       |
| F03 | Widget trong chat  | Quiz trắc nghiệm mặc định, listening widget, speaking widget, recap/paywall widget.  |
| F04 | Kho quiz learner   | Danh sách bộ quiz, làm bài, audio listening, ghi transcript speaking, xem AI review. |
| F05 | Quiz cá nhân hóa   | Tạo quiz ôn lỗi từ error_logs và kết quả sai trong các quiz trước.                   |
| F06 | Thi nâng cấp       | Bài nội bộ text/listening/speaking để mở level tiếp theo khi đủ điểm.                |

|     |         |                                                                                               |
|-----|---------|-----------------------------------------------------------------------------------------------|
| F07 | Billing | Gói Free/Plus/Ultra, quota chat/quiz, yêu cầu thanh toán QR, admin duyệt.                     |
| F08 | Admin   | Dashboard, user CRUD, chỉnh CEFR/gói/role, quiz CRUD/import/source/PDF/image, payment review. |

### 2.5. Kiến trúc tổng thể

Hệ thống được tách thành frontend, API, agent worker, LiveKit và database. Frontend không giữ secret. API cấp JWT/refresh token, cấp LiveKit token và quản lý nghiệp vụ. Agent worker nhận job từ LiveKit, đọc metadata, tạo LangGraph và tham gia room như một participant.



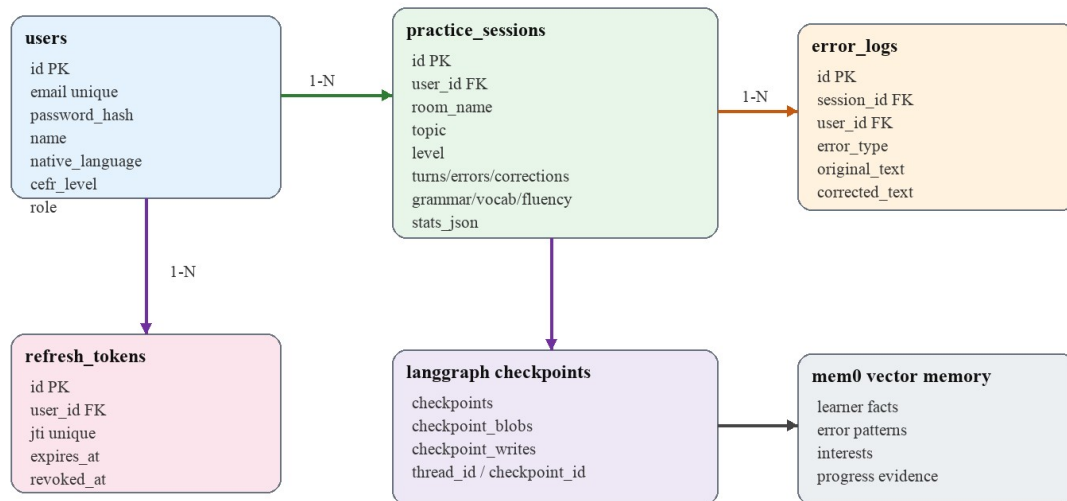
Hình 2.1. Kiến trúc tổng thể EnglishAltutor

### 2.6. Mô hình dữ liệu

Các bảng lỗi gồm users, refresh\_tokens, password\_reset\_codes, practice\_sessions, error\_logs, quizzes, quiz\_sets, quiz\_attempts và payment\_requests. Dữ liệu này giúp nổi phiên học với bài tập và kết quả, đồng thời phục vụ dashboard và review.



## Core Database Model



Hình 2.2. Mô hình dữ liệu lõi

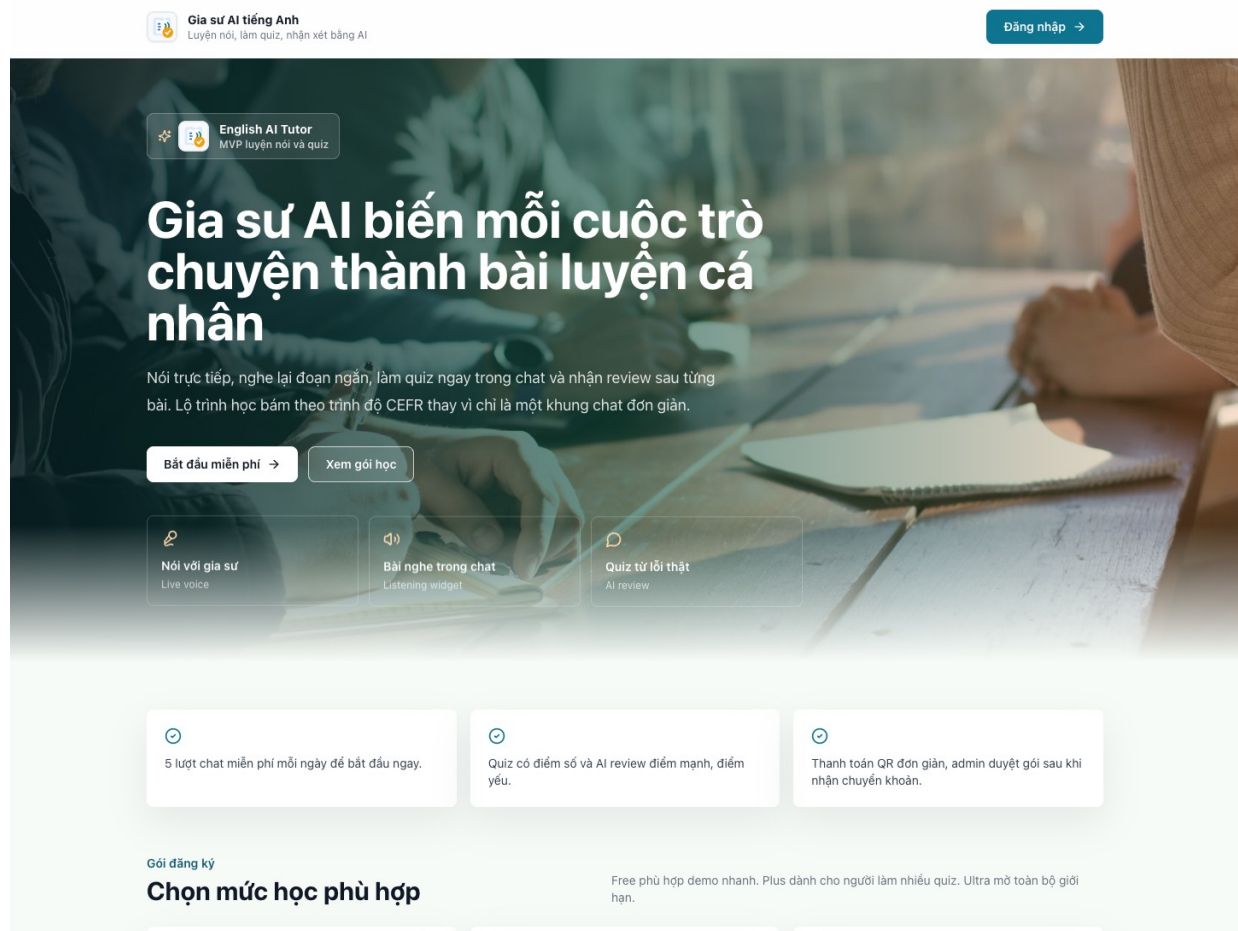
## CHƯƠNG 3. THIẾT KẾ GIAO DIỆN VÀ LỒNG TRẢI NGHIỆM

### 3.1. Nguyên tắc UI/UX

Giao diện được Việt hóa để giảm rào cản cho người dùng phổ thông. Learner chỉ nhìn thấy chức năng học: học tập, Gia sư AI, ôn lỗi, bài quiz, gói học và tiến độ. Admin có luồng riêng để quản trị hệ thống, tránh trộn công cụ tạo quiz vào màn người học.

### 3.2. Landing page và upsell

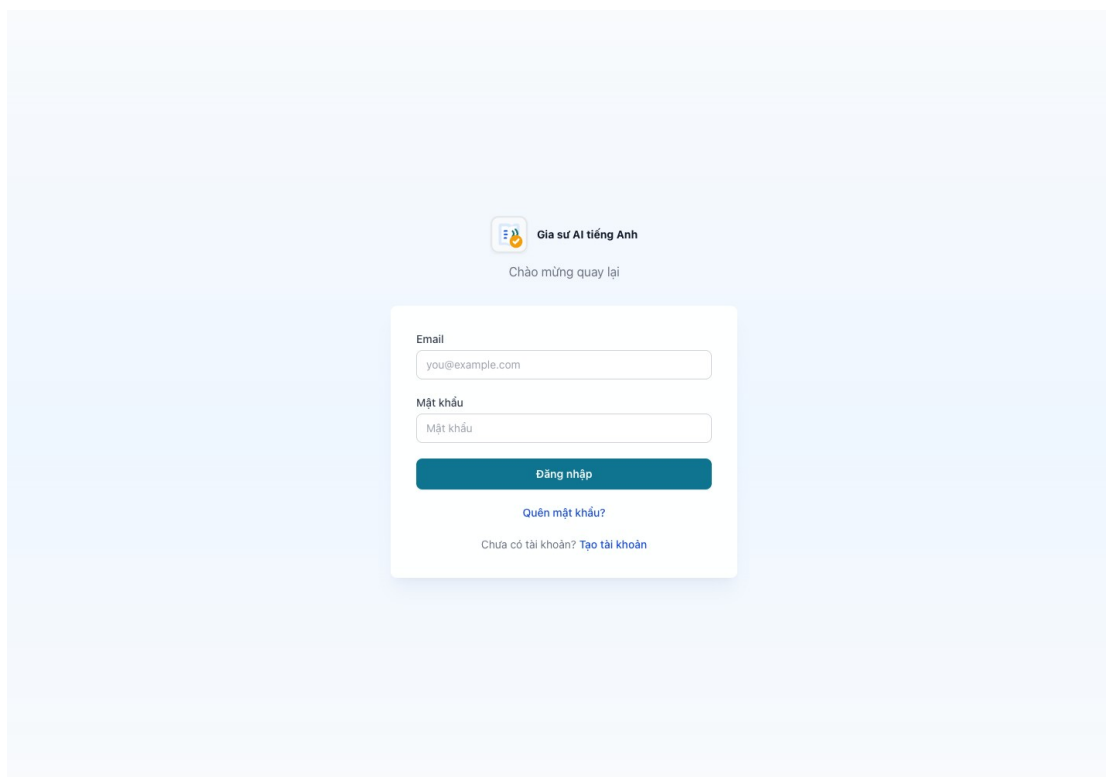
Landing page giải thích rõ giá trị: nói với gia sư AI, làm quiz trong chat, nhận review và chọn gói. Ba gói học được nêu ngay trên trang: Free, Plus và Ultra.



Hình 3.1. Landing page với thông điệp sản phẩm và gói học

### 3.3. Auth và quên mật khẩu

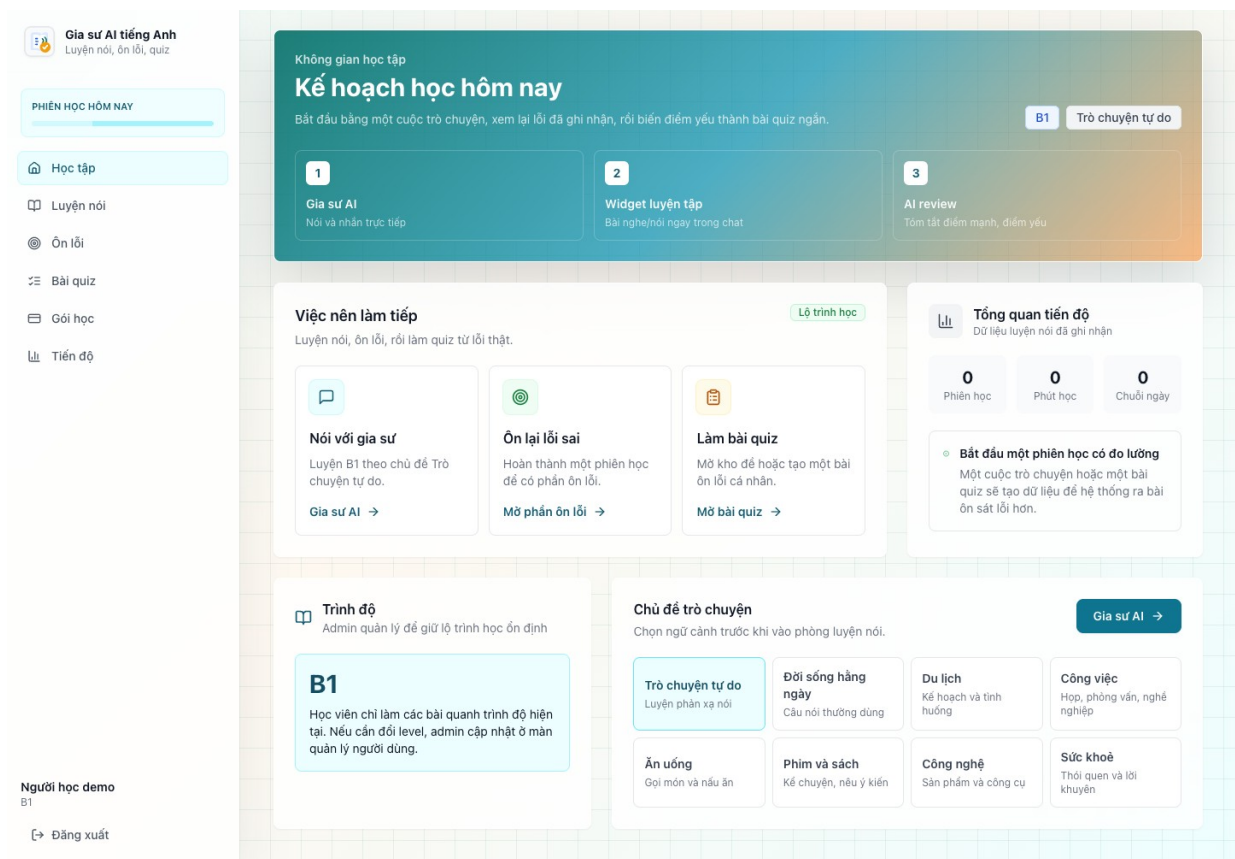
Màn auth hỗ trợ đăng nhập, tạo tài khoản và quên mật khẩu bằng mã gửi về email. Tài khoản mới được khởi tạo ở B1; admin có quyền thay đổi CEFR.



Hình 3.2. Màn đăng nhập/đăng ký/quên mật khẩu

### 3.4. Trang học tập của learner

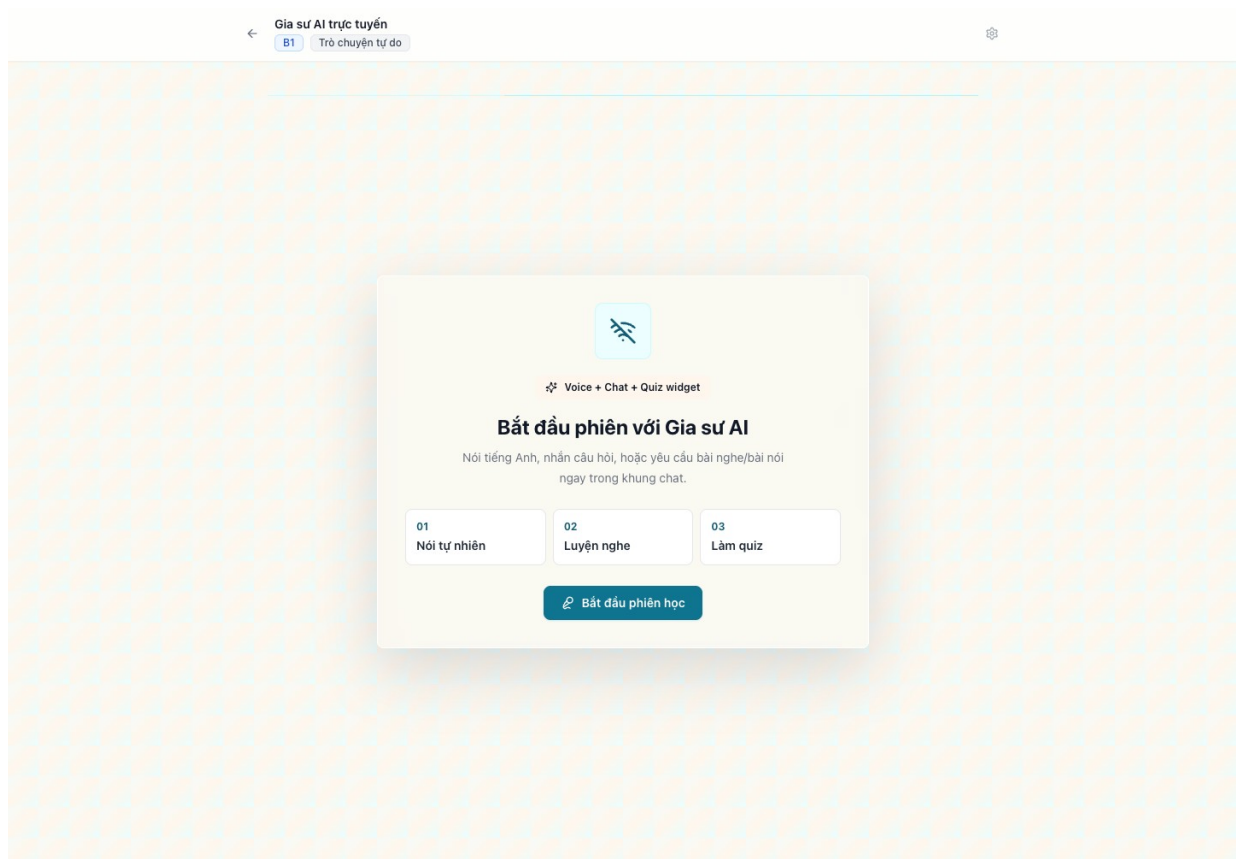
Home đóng vai trò dashboard thao tác nhanh: vào Gia sư AI, xem điểm yếu, làm quiz, xem gói và tiến độ. Learner không tự chọn level; level hiển thị theo hồ sơ.



Hình 3.3. Trang học tập của learner

### 3.5. Gia sư AI trực tuyến

Practice room là nơi learner nói hoặc gõ với tutor. Khung chat có thể nhận transcript, hiển thị tin nhắn voice/text, phát TTS của agent, nhận quiz widget/listening/speaking widget và hiện popup khi hết lượt.



Hình 3.4. Màn Gia sư AI sẵn sàng bắt đầu phiên

### 3.6. Kho quiz và bài thi nâng cấp

Kho quiz được tổ chức theo bộ đề, có level, topic, nguồn và số câu. Learner làm được các bài quanh level hiện tại, có bài ôn lỗi cá nhân và bài thi nâng cấp.

Gia sư AI tiếng Anh

Luyện nói, ôn lỗi, quiz

PHIÊN HỌC HÔM NAY

Học tập

Luyện nói

Ôn lỗi

Bài quiz

Gói học

Tiến độ

Người học demo

B1

Đăng xuất

Kho bài luyện

Làm quiz tiếng Anh

Chọn một bài trong kho để đo quản trị viên chuẩn bị, làm bài và xem AI nhận xét điểm mạnh, điểm yếu.

Tải lại

Bài quiz có thể làm

48

Tổng số câu hỏi

240

Điểm gần nhất

N/A

Quiz ôn lỗi cá nhân

Tạo một bài ngắn bám vào lỗi nói, lỗi quiz gần đây và nhóm kỹ năng bạn còn yếu.

Tạo quiz ôn lỗi

Thi nâng cấp trình độ

B1 → B2

Làm bài kiểm tra nội bộ từ B1 lên B2. Cần đạt tối thiểu 80% tổng điểm và không dưới 65% ở phần nghe/nói. Bài gồm quiz chữ, nghe audio và nói qua micro; đây là đánh giá nội bộ, không thay thế chứng chỉ quốc tế.

Bắt đầu thi nâng cấp

Danh sách quiz

Quiz được gom theo bộ đề và lọc theo trình độ hiện tại của bạn.

9 bộ · 48 bài

CEFR C1 Listening & Speaking Bank

Nguồn mở

C1

communication skills · 8 bài · 40 câu

Ngân hàng câu hỏi nghe/nói C1 dùng cho quiz và widget trong Gia sư AI. Các câu hỏi được tạo mới theo CEFR can-do, không sao chép để thi chính thức.

C1 Listening - Đời Sống Hằng Ngày

C1 · 5 câu · 3 thg 5

Làm quiz →

C1 Speaking - Đời Sống Hằng Ngày

C1 · 5 câu · 3 thg 5

Làm quiz →

C1 Listening - Học Tập

C1 · 5 câu · 3 thg 5

Làm quiz →

C1 Speaking - Học Tập

C1 · 5 câu · 3 thg 5

Làm quiz →

C1 Listening - Công Việc

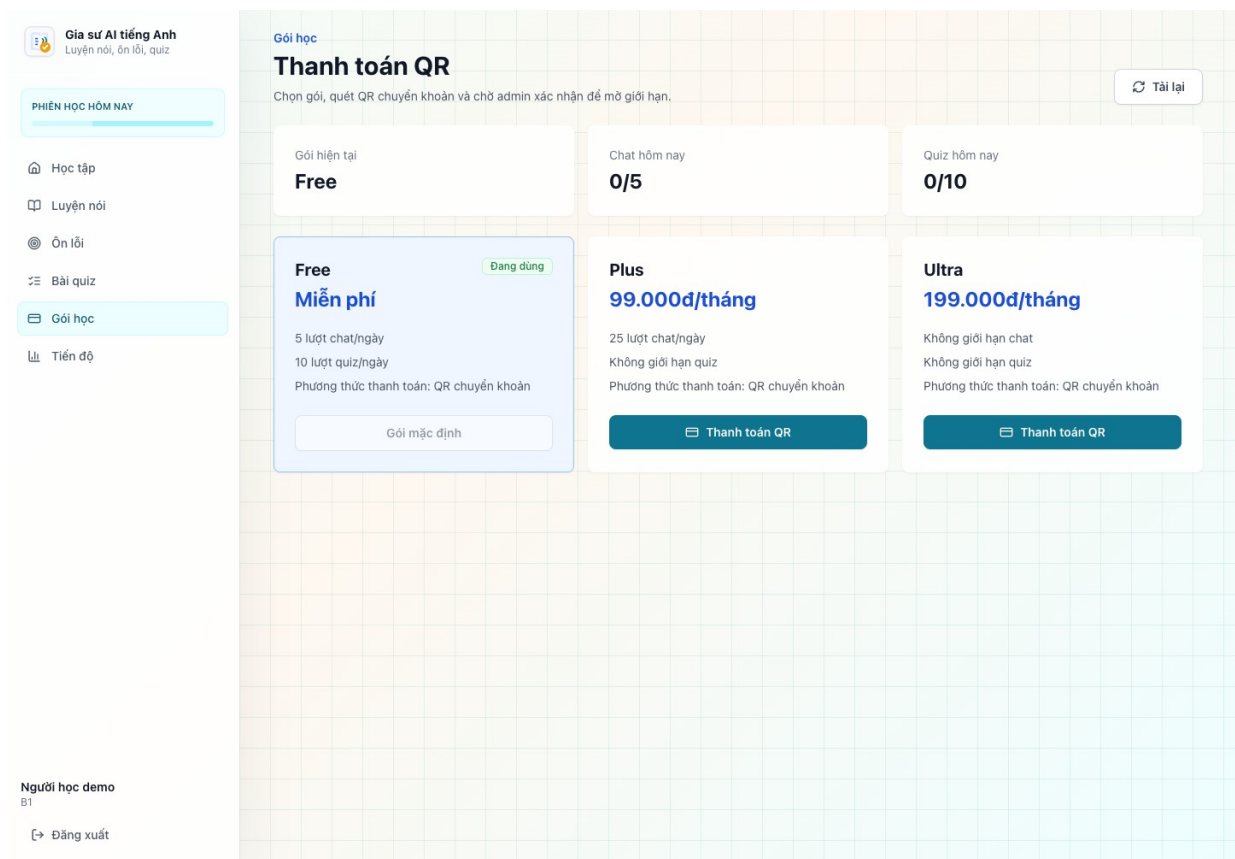
C1 · 5 câu · 3 thg 5

Làm quiz →

Hình 3.5. Kho quiz theo bộ và bài thi nâng cấp

### 3.7. Billing QR

Màn billing hiển thị gói hiện tại, số lượt đã dùng trong ngày, các gói có thể nâng cấp và QR chuyển khoản. Sau khi người dùng tạo yêu cầu, admin duyệt để mở gói.



Hình 3.6. Màn thanh toán QR và quota sử dụng

## CHƯƠNG 4. XÂY DỰNG HỆ THỐNG

### 4.1. Frontend route và phân quyền

Frontend định nghĩa LearnerRoute, AdminRoute và GuestRoute. Người dùng chưa đăng nhập đi tới landing/login; learner vào app học; admin được điều hướng sang khu quản trị. Cấu trúc route nằm ở frontend/src/App.tsx.

### 4.2. Auth, JWT và refresh token

Backend sử dụng access JWT ngắn hạn có claim sub, role, type, exp và iat. Refresh token có JTI, được lưu trong bảng refresh\_tokens và đặt trong httpOnly cookie. Khi refresh, token cũ bị revoke và phát hành token mới, giảm rủi ro reuse.

#### Access JWT và refresh token có JTI

backend/app/core/auth.py:61-86

```
61 def create_access_token(user: User) -> str:
62     issued_at = _utc_now_aware()
63     expire = issued_at + timedelta(minutes=settings.JWT_EXPIRE_MINUTES)
64     payload = {
65         "sub": user.id,
66         "role": user.role,
67         "type": "access",
68         "exp": expire,
69         "iat": issued_at,
70     }
71     return jwt.encode(payload, settings.JWT_SECRET_KEY, algorithm=settings.JWT_ALGORITHM)
72
73
74 def create_refresh_token(user_id: str) -> tuple[str, str, datetime]:
75     issued_at = _utc_now_aware()
76     expires_at = issued_at + timedelta(days=settings.REFRESH_TOKEN_EXPIRE_DAYS)
77     jti = uuid4().hex
78     payload = {
79         "sub": user_id,
80         "jti": jti,
81         "type": "refresh",
82         "exp": expires_at,
83         "iat": issued_at,
84     }
85     token = jwt.encode(payload, settings.JWT_SECRET_KEY, algorithm=settings.JWT_ALGORITHM)
86     return token, jti, expires_at.replace(tzinfo=None)
```

Hình 4.1. Code tạo access JWT và refresh token

### 4.3. Gói học và kiểm soát quota

Gói Free có 5 lượt chat/ngày và 10 lượt quiz/ngày. Plus có 25 lượt chat và không giới hạn quiz. Ultra không giới hạn toàn bộ. Token API và submit quiz gọi `assert_quota_available` để chặn khi hết lượt.



#### Gói học và giới hạn lượt sử dụng

backend/app/core/subscriptions.py:29-106

```
29 PLANS: dict[str, Plan] = {
30     "free": Plan(
31         code="free",
32         name="Free",
33         price_vnd=0,
34         chat_limit=5,
35         quiz_limit=10,
36         description="5 lượt chat mỗi ngày và 10 lượt làm quiz mỗi ngày.",
37     ),
38     "plus": Plan(
39         code="plus",
40         name="Plus",
41         price_vnd=99_000,
42         chat_limit=25,
43         quiz_limit=None,
44         description="25 lượt chat mỗi ngày và không giới hạn làm quiz.",
45     ),
46     "ultra": Plan(
47         code="ultra",
48         name="Ultra",
49         price_vnd=199_000,
50         chat_limit=None,
51         quiz_limit=None,
52         description="Không giới hạn toàn bộ.",
53     ),
54 }
55
56
57 def normalize_plan(plan: str | None) -> Plan:
58     return PLANS.get((plan or "free").strip().lower(), PLANS["free"])
59
60
61 def today_start() -> datetime:
62     now = datetime.utcnow()
63     return now.replace(hour=0, minute=0, second=0, microsecond=0)
64
65
66 async def usage_today(user_id: str) -> dict[str, int]:
67     start = today_start()
68     factory = get_session_factory()
69     async with factory() as session:
70         chat_count = int(
71             (
72                 await session.execute(
73                     select(func.count(PracticeSession.id)).where(
74                         PracticeSession.user_id == user_id,
75                         PracticeSession.started_at >= start,
76                     )
77                 )
78             ).scalar_one()
79         )
80         quiz_count = int(
81             (
82                 await session.execute(
83                     select(func.count(QuizAttempt.id)).where(
84                         QuizAttempt.user_id == user_id,
85                         QuizAttempt.created_at >= start,
86                     )
87                 )
88             ).scalar_one()
89         )
90     return {"chat": chat_count, "quiz": quiz_count}
91
92
93 async def assert_quota_available(user: User, kind: UsageKind) -> None:
94     plan = normalize_plan(user.subscription_plan)
95     limit = plan.chat_limit if kind == "chat" else plan.quiz_limit
96     if limit is None:
97         return
98
99     usage = await usage_today(user.id)
100     used = usage[kind]
101     if used >= limit:
102         label = "chat với gia sư AI" if kind == "chat" else "làm quiz"
103         raise HTTPException(
104             status_code=status.HTTP_402_PAYMENT_REQUIRED,
105             detail=f"Gói {plan.name} đã hết {label} hôm nay ({used}/{limit}). Hãy nâng cấp gói tiếp tục.",
106         )
```

Hình 4.2. Code định nghĩa gói học và kiểm tra quota

## 4.4. Cấp LiveKit token và dispatch agent

Khi learner bắt đầu phiên, API tạo room token có metadata user\_id, topic và level. RoomConfiguration gắn AgentDispatch để LiveKit gọi agent worker vào đúng phòng. Frontend không biết LiveKit secret.

#### AgentSession tích hợp LangGraph, EdgeTTS, WhisperSTT và VAD

backend/app/agents/english\_tutor/entrypoint.py:289-335

```
289 # — Initialize memory —
290 memory_mgr = await _get_memory_manager()
291 memory_prompt = "No previous memories about this learner."
292 memory_client_raw = None
293
294 if memory_mgr:
295     try:
296         memory_prompt = await memory_mgr.search_memories(
297             query=f"English learner profile, level {level}, topic {topic}",
298             user_id=user_id,
299             limit=5,
300         )
301         memory_client_raw = memory_mgr.client
302     except Exception as e:
303         logger.warning(f"Memory search failed: {e}")
304
305 # — Build LangGraph —
306 thread_id = str(uuid4())
307 graph = await create_english_tutor_graph(use_checkpoint=True)
308
309 graph_config = {
310     "configurable": {"thread_id": thread_id},
311     "metadata": {
312         "user_id": user_id,
313         "topic": topic,
314         "level": level,
315         "memory_prompt": memory_prompt,
316         "memory_client": memory_client_raw,
317         "greeting_message": "",
318     },
319 }
320
321 # — Create agent session with TTS + STT + LLM —
322 session = AgentSession(
323     llm=LangGraphAdapter(graph=graph, config=graph_config),
324     tts=EdgeTTS(voice="en-US-JennyNeural"),
325     stt=WhisperSTT(model_size="base.en"),
326     vad=silero.VAD.load(
327         min_speech_duration=float_env("VAD_MIN_SPEECH_DURATION", 0.15),
328         min_silence_duration=float_env("VAD_MIN_SILENCE_DURATION", 1.05),
329         prefix_padding_duration=float_env("VAD_PREFIX_PADDING_DURATION", 0.55),
330         activation_threshold=float_env("VAD_ACTIVATION_THRESHOLD", 0.55),
331     ),
332     turn_detection="vad",
333     min_endpointing_delay=float_env("AGENT_MIN_ENDPOINTING_DELAY", 1.1),
334     max_endpointing_delay=float_env("AGENT_MAX_ENDPOINTING_DELAY", 4.5),
335     preemptive_generation=False,
```

Hình 4.3. AgentSession tích hợp LangGraph, TTS, STT và VAD

## 4.5. WebRTC trên frontend

Hook useLiveKit chịu trách nhiệm kết nối room, gắn audio track của agent, nhận transcription, chống duplicate transcript, gửi text stream và nhận quiz/chat widget. Đây là phần quan trọng để giải thích vì sao voice, chat và widget nằm trong cùng phiên.

## Frontend nhận transcript và widget qua LiveKit

frontend/src/hooks/useLiveKit.ts:282-370

```
282   room.on(RoomEvent.TranscriptionReceived, (segments, participant) => {
283     for (const segment of segments) {
284       const text = segment.text?.trim();
285       if (!text) continue;
286
287       const isLocal =
288         participant?.identity === room.localParticipant.identity;
289       const role = isLocal ? "user" : "assistant";
290       const label = isLocal ? "Bạn" : "Gia sư";
291
292       if (segment.final) {
293         setCurrentTranscript("");
294         if (role === "assistant") {
295           setAgentReady(true);
296         }
297         queueTranscriptMessage(role, text, segment.id);
298       } else {
299         setCurrentTranscript(`${label}: ${text}`);
300       }
301     }
302   });
303
304   // Text streams on this topic are used for typed user input. Older backend
305   // builds also sent assistant text here, so drain remote streams and let
306   // the final LiveKit transcription be the only assistant bubble source.
307   room.registerTextStreamHandler(CHAT_TOPIC, async (reader) => {
308     const textStream = reader as any;
309     if (typeof textStream.readAll === "function") {
310       await textStream.readAll().catch(() => "");
311     } else if (typeof textStream[Symbol.asyncIterator] === "function") {
312       for await (const chunk of textStream) {
313         void chunk;
314         // Drain the stream so LiveKit can close it cleanly.
315       }
316     }
317   });
318
319   room.registerTextStreamHandler(QUIZ_WIDGET_TOPIC, async (reader) => {
320     try {
321       const textStream = reader as any;
322       const raw =
323         typeof textStream.readAll === "function"
324           ? await textStream.readAll()
325           : await (async () => {
326               let merged = "";
327               if (typeof textStream[Symbol.asyncIterator] === "function") {
328                 for await (const chunk of textStream) {
329                   merged += String(chunk ?? "");
330                 }
331               }
332               return merged;
333             })();
334
335       const payload = JSON.parse(raw || "{}") as InlineQuizWidget;
336       const hasExerciseQuestions = Array.isArray(payload?.questions) && payload.questions.length > 0;
337       const hasLegacyQuestion = Array.isArray(payload?.choices) && payload.choices.length > 0;
338       if (!payload?.id || (!hasExerciseQuestions && !hasLegacyQuestion)) return;
339       addQuizWidget(payload);
340     } catch (error) {
341       console.error("Failed to read quiz widget stream:", error);
342     }
343   });
344
345   room.registerTextStreamHandler(CHAT_WIDGET_TOPIC, async (reader) => {
346     try {
347       const textStream = reader as any;
348       const raw =
349         typeof textStream.readAll === "function"
350           ? await textStream.readAll()
351           : await (async () => {
352               let merged = "";
353               if (typeof textStream[Symbol.asyncIterator] === "function") {
354                 for await (const chunk of textStream) {
355                   merged += String(chunk ?? "");
356                 }
357               }
358               return merged;
359             })();
360
361       const payload = JSON.parse(raw || "{}") as ChatWidget;
362       if (!payload?.id || !payload?.type || !payload?.title) return;
363       addChatWidget(payload);
364     } catch (error) {
365       console.error("Failed to read chat widget stream:", error);
366     }
367   });
368
369   await room.connect(getLiveKitUrl(), token, {
370     rtcConfig: getRtcConfig(),
```

Hình 4.4. Frontend nhận transcript và widget qua LiveKit

## 4.6. System prompt và LangGraph

System prompt định nghĩa vai trò tutor, CEFR, topic, memory, style phản hồi và kỹ thuật sửa lỗi recast. LangGraphAdapter đưa graph vào LiveKit AgentSession, còn các node assess/respond/correct xử lý từng lượt.

```
System prompt điều chỉnh theo CEFR và phong cách sửa lỗi
backend/app/agents/english_tutor/prompts.py:5-45

5 ENGLISH_TUTOR_PROMPT = """\
6 You are a friendly, patient English conversation tutor. Your goal is to help the user \
7 practice and improve their English through natural, engaging conversation.
8
9 ## User Profile
10 - **CEFR Level:** {level}
11 - **Current Topic:** {topic}
12 - **Today's Date:** {date}
13
14 ## Relevant Memories (from previous sessions)
15 {memory_prompt}
16
17 ## Current Tutoring Context
18 {session_context}
19
20 ## Core Behavior
21 1. **Be conversational** – talk like a supportive friend, not a textbook. Use natural \
22 English appropriate for the user's level.
23 2. **Adapt your language** – For A1-A2: use simple words, short sentences. For B1-B2: \
24 normal conversation. For C1-C2: use idioms, complex structures, nuanced vocabulary.
25 3. **Gently correct** – When you notice an error, don't stop the flow. Naturally rephrase \
26 what the user said correctly in your response (recasting technique). Only give explicit \
27 corrections for repeated or important errors.
28 4. **Encourage speaking** – Ask follow-up questions. Show genuine interest. React to what \
29 they say before asking something new.
30 5. **Stay on topic** – but follow the user's lead if they want to change subjects.
31
32 ## Response Guidelines
33 - Keep responses **2-4 sentences** for voice conversation (concise for spoken English).
34 - Use **natural fillers** occasionally ("Well,", "You know,", "That's interesting!") to \
35 sound human.
36 - **Never** switch to the user's native language.
37 - **Never** say you're an AI or break character.
38 - If the user seems stuck, help them with a hint or rephrase your question in simpler terms.
39
40 ## Correction Style (Recasting)
41 When the user says something incorrect, weave the correction into your response naturally:
42 - User: "I goed to the store yesterday"
43 - You: "Oh, you **went** to the store? What did you buy?"
44 (Don't say: "The correct form is 'went', not 'goed'." – unless they keep making the same mistake.)
45 """
```

Hình 4.5. System prompt chính của gia sư AI

## 4.7. Widget quiz/listening/speaking trong chat

Agent có thể gửi payload widget qua LiveKit text stream. Khi learner nói 'cho tôi làm bài tập', hệ thống mặc định tạo quiz trắc nghiệm thường. Nếu learner nói rõ 'luyện nghe' hoặc 'luyện nói', agent tạo widget tương ứng.

# Agent gửi quiz/widget qua LiveKit text stream

backend/app/agents/english\_tutor/endpoint.py:425-480

```

425     async def send_quiz_widget(payload: dict):
426         writer = await ctx.room.local_participant.stream_text(topic=FE_QUIZ_WIDGET_TOPIC)
427         await writer.write(json.dumps(payload, ensure_ascii=False))
428         await writer.aclose()
429
430     async def send_chat_widget(payload: dict):
431         writer = await ctx.room.local_participant.stream_text(topic=FE_CHAT_WIDGET_TOPIC)
432         await writer.write(json.dumps(payload, ensure_ascii=False))
433         await writer.aclose()
434
435     async def emit_exercise_set(
436         state_values: dict,
437         turn_number: int,
438         request_text: str | None = None,
439         exercise_mode: str = "auto",
440     ) -> bool:
441         nonlocal exercise_widget_count
442         nonlocal last_widget_turn
443         nonlocal last_widget_request_key
444         nonlocal last_widget_sent_at
445
446         request_key = f"{turn_number}:{_normalize_intent_text(request_text)[:120]}"
447         if request_key == last_widget_request_key and time.time() - last_widget_sent_at < 8:
448             return False
449
450         if not request_text and turn_number == last_widget_turn:
451             return False
452
453         sequence = exercise_widget_count + 1
454         payload = await build_inline_exercise_set_from_bank(
455             state_values,
456             request_text=request_text,
457             exercise_mode=exercise_mode,
458             sequence=sequence,
459         )
460         if not payload:
461             payload = build_inline_exercise_set(
462                 state_values,
463                 request_text=request_text,
464                 exercise_mode=exercise_mode,
465                 sequence=sequence,
466             )
467         if not payload:
468             return False
469
470         await send_quiz_widget(payload)
471         exercise_widget_count = sequence
472         last_widget_turn = turn_number
473         last_widget_request_key = request_key
474         last_widget_sent_at = time.time()
475         logger.info(
476             "Inline exercise set emitted for turn {} with mode {}",
477             turn_number,
478             payload.get("mode"),
479         )
480     return True

```

Hình 4.6. Agent gửi widget vào room

#### Trigger bài tập trong chat theo intent của người học

backend/app/agents/english\_tutor/endpoint.py:503-548

```
503     explicit_request = _wants_exercise(user_text)
504     requested_mode = _exercise_mode(user_text)
505     logger.info(
506         "Exercise flow check: turn={}, explicit_request={}, mode={}, text={}",
507         turn_number,
508         explicit_request,
509         requested_mode,
510         user_text[:80],
511     )
512
513     if exercise_offer_pending:
514         if _is_negative(user_text):
515             exercise_offer_pending = False
516             return False
517
518         if _is_affirmative(user_text) or explicit_request:
519             emitted = await emit_exercise_set(
520                 state_values,
521                 turn_number,
522                 user_text,
523                 requested_mode if explicit_request else "auto",
524             )
525             exercise_offer_pending = False
526             if emitted:
527                 await session.say(
528                     f"({_exercise_reply(requested_mode if explicit_request else 'auto')} Try it now.",
529                     allow_interruptions=False,
530                 )
531             return emitted
532
533     if turn_number - exercise_offer_turn >= 3:
534         exercise_offer_pending = False
535
536     if explicit_request:
537         emitted = await emit_exercise_set(
538             state_values,
539             turn_number,
540             user_text,
541             requested_mode,
542         )
543         if emitted:
544             await session.say(
545                 _exercise_reply(requested_mode),
546                 allow_interruptions=False,
547             )
548     return emitted
```

Hình 4.7. Trigger tạo bài tập trong chat

## Widget quiz lấy câu từ ngân hàng trong database

backend/app/agents/english\_tutor/inline\_quiz.py:228-320

```
228 async def build_inline_exercise_set_from_bank(
229     state: EnglishTutorState,
230     *,
231     request_text: str | None = None,
232     exercise_mode: str = "auto",
233     sequence: int = 0,
234 ) -> dict[str, Any] | None:
235     """Build an inline widget from the real quiz bank stored in Postgres."""
236     source_text = _last_user_text(state)
237     topic = _topic_display(state)
238     level = _clean(state.get("working_level") or state.get("user_level")) or "B1"
239     selected_mode = exercise_mode
240     if selected_mode == "auto":
241         selected_mode = "grammar"
242
243     seed_text = "|".join([topic, level, source_text[:120], request_text or "", str(sequence)])
244     candidate_levels = _candidate_levels(level)
245
246     try:
247         factory = get_session_factory()
248         async with factory() as db:
249             result = await db.execute(
250                 select(Quiz)
251                 .where(
252                     Quiz.source != "level_test",
253                     Quiz.level.in_(candidate_levels),
254                 )
255                 .order_by(desc(Quiz.created_at))
256                 .limit(BANK_QUERY_LIMIT)
257             )
258             quizzes = list(result.scalars().all())
259     except Exception as exc:
260         logger.warning(f"Could not load inline exercise bank: {exc}")
261         return None
262
263     quizzes.sort(
264         key=lambda quiz: (
265             0 if (quiz.topic or "") == topic else 1,
266             _level_distance(level, quiz.level or ""),
267             0 if quiz.source in {"open_source", "book", "imported"} else 1,
268             quiz.title or "",
269         )
270     )
271
272     grammar_pool: list[dict[str, Any]] = []
273     listening_pool: list[dict[str, Any]] = []
274     speaking_pool: list[dict[str, Any]] = []
275     for quiz in quizzes:
276         for index, raw_question in enumerate(quiz.questions_json or [], start=1):
277             question_type = _clean(raw_question.get("type") or raw_question.get("question_type"))
278             converted = _inline_question_from_bank(
279                 raw_question,
280                 question_id=f"bank-{quiz.id}-{index}",
281             )
282             if not converted:
283                 continue
284             if question_type.startswith("listening"):
285                 listening_pool.append(converted)
286             elif question_type == "speaking_prompt":
287                 speaking_pool.append(converted)
288             elif question_type == "multiple_choice":
289                 grammar_pool.append(converted)
290
291     if selected_mode == "listening":
292         questions = _take_rotated(listening_pool, sequence=sequence, seed_text=seed_text, count=2)
293     elif selected_mode == "speaking":
294         questions = _take_rotated(speaking_pool, sequence=sequence, seed_text=seed_text, count=2)
295     elif selected_mode == "grammar":
296         questions = _take_rotated(grammar_pool, sequence=sequence, seed_text=seed_text, count=3)
297     else:
298         questions = [
299             *_take_rotated(grammar_pool, sequence=sequence, seed_text=f"{seed_text}|g", count=1),
300             *_take_rotated(listening_pool, sequence=sequence, seed_text=f"{seed_text}|l", count=1),
301             *_take_rotated(speaking_pool, sequence=sequence, seed_text=f"{seed_text}|s", count=1),
302         ]
303
304     minimum = 2 if selected_mode in {"listening", "speaking"} else 1
305     if len(questions) < minimum:
306         logger.info(
307             "Inline bank has insufficient questions for mode {}: grammar={}, listening={}, speaking={}",
308             selected_mode,
309             len(grammar_pool),
310             len(listening_pool),
311             len(speaking_pool),
312         )
313         return None
314
315     return _bank_widget_payload(
316         mode=selected_mode,
317         topic=topic,
318         level=level,
319         questions=questions,
320     )
```



Hình 4.8. Widget lấy câu hỏi từ ngân hàng quiz trong database

## 4.8. Kho quiz, import nguồn và import sách PDF

Admin có thể tạo quiz thủ công, import JSON, sinh bộ quiz từ nguồn tham khảo như CEFR, Wikibooks, Tatoeba, format THPT 2025 hoặc đọc PDF sách để tạo câu hỏi mới. Luồng PDF chỉ dùng đoạn trích ngắn để tạo câu hỏi gốc, không sao chép nguyên văn bài tập bản quyền.

Admin tạo bộ quiz từ nguồn tham khảo chuẩn

```

1297 async def generate_source_quizzes(req: QuizImportRequest, info: dict, source_text: str) ->
1298     list[QuizImportItem]:
1299         if not (source_text or "").strip():
1300             raise HTTPException(
1301                 status_code=422,
1302                 detail=(
1303                     f"Không đọc được nội dung từ ngu[n {info['title']}). "
1304                     "Hãy th[i] URL khác, gi[i]m b[o]t mục tiêu, hoặc dùng import file đ[i] đưa đ[i] thật vào hệ th[i]ng."
1305                 ),
1306             )
1307         topic_display = req.topic.replace("_", " ")
1308         prompt = f"""
1309 Create a batch of original English learner quizzes for this app.
1310
1311 Source preset: {info['title']}
1312 Source URL: {info.get('url')} or 'not provided'
1313 License note: {info['license']}
1314 Attribution: {info['attribution']}
1315 Source guidance: {info['guidance']}
1316
1317 Target:
1318 - CEFR level: {req.level}
1319 - Topic: {topic_display}
1320 - Focus: {req.focus or 'balanced grammar, vocabulary, reading comprehension, and practical communication'}
1321 - Quiz count: {req.quiz_count}
1322 - Questions per quiz: {req.questions_per_quiz}
1323
1324 Rules:
1325 - Generate original questions; do not copy long copyrighted passages verbatim.
1326 - If the source text is open licensed, still keep prompts concise and add attribution in quiz description.
1327 - Every quiz must contain exactly {req.questions_per_quiz} questions.
1328 - Use only multiple_choice and fill_blank.
1329 - For multiple_choice, provide exactly 4 options and one correct_answer that exactly matches one option.
1330 - For fill_blank, provide no options and a short correct_answer.
1331 - Keep questions useful for Vietnamese learners of English.
1332 - Use focus labels like grammar, vocabulary, word_choice, structure, comprehension, speaking.
1333 - Do not return generic placeholder questions.
1334 - Every quiz should contain concrete, source-grounded language practice.
1335
1336 Source excerpt:
1337 {source_text}
1338
1339 Return structured data only.
1340 """
1341         try:
1342             llm = get_model(settings.DEFAULT_MODEL, temperature=0.25).with_structured_output(GeneratedQuizBatch)
1343             batch: GeneratedQuizBatch = await llm.ainvoke(
1344                 [
1345                     SystemMessage(content="You create reliable, original English learning quizzes with CEFR-aligned
1346 difficulty."),
1347                     HumanMessage(content=prompt),
1348                 ]
1349             )

```

Hình 4.9. Sinh quiz từ nguồn tham khảo chuẩn



```
Import PDF sách và sinh quiz an toàn bản quyền
backend/app/router/quizzes.py:1391-1445

1391 async def _generate_book_quizzes(
1392     *,
1393     book_title: str,
1394     page_range: str,
1395     book_text: str,
1396     topic: str,
1397     level: str,
1398     quiz_count: int,
1399     questions_per_quiz: int,
1400     focus: str | None,
1401 ) -> list[QuizImportItem]:
1402     topic_display = topic.replace("_", " ")
1403     prompt = f"""
1404     Create a batch of original English learner quizzes from a user-uploaded grammar/IELTS book excerpt.
1405
1406     Source book: {book_title}
1407     Pages used: {page_range}
1408
1409     Target:
1410     - CEFR level: {level}
1411     - Topic: {topic_display}
1412     - Focus: {focus or 'grammar accuracy, IELTS-style language use, vocabulary, and sentence structure'}
1413     - Quiz count: {quiz_count}
1414     - Questions per quiz: {questions_per_quiz}
1415
1416     Rules:
1417     - Generate original questions inspired by the grammar points and skills in the excerpt.
1418     - Do not copy long passages, examples, answer keys, or exercises from the book.
1419     - Keep any quoted phrase extremely short when needed.
1420     - Every quiz must contain exactly {questions_per_quiz} questions.
1421     - Use only multiple_choice and fill_blank.
1422     - For multiple_choice, provide exactly 4 options and one correct answer that exactly matches one option.
1423     - For fill_blank, provide no options and a short correct answer.
1424     - Explanations must be in Vietnamese and tell the learner what grammar point to notice.
1425     - Questions should be useful for Vietnamese learners preparing for IELTS or school English.
1426     - Do not return generic placeholder questions.
1427
1428     Book excerpt:
1429     {book_text}
1430
1431     Return structured data only.
1432     """
1433     try:
1434         llm = get_model(settings.DEFAULT_MODEL, temperature=0.22).with_structured_output(GeneratedQuizBatch)
1435         batch: GeneratedQuizBatch = await llm.ainvoke(
1436             [
1437                 SystemMessage(
1438                     content=(
1439                         "You create copyright-safe, original English quizzes from limited book excerpts. "
1440                         "You never reproduce full copyrighted exercises."
1441                     ),
1442             ),
1443             HumanMessage(content=prompt),
1444         ]
1445     )
```

Hình 4.10. Sinh quiz từ PDF sách theo hướng an toàn bản quyền

4.9. AI review kết quả và quiz ôn lỗi

Sau khi nộp quiz, backend chấm từng câu, lưu quiz\_attempts và trả review theo focus. Với speaking\_prompt, hệ thống dùng transcript để đánh giá mức hoàn thành theo rubric. Quiz ôn lỗi cá nhân được sinh từ error\_logs và các câu từng làm sai, giúp sản phẩm phục vụ đúng bài toán sửa lỗi.

4.10. Admin vận hành

Admin quản lý người dùng, chỉnh CEFR, đổi gói, khóa/xóa user, quản lý kho quiz, sửa/xóa quiz và duyệt payment request. Đây là phần làm sản phẩm vận hành được trong bối cảnh demo, không chỉ là app cá nhân.

CHƯƠNG 5. KIỂM THỬ, ĐÁNH GIÁ VÀ KẾ HOẠCH DEMO

5.1. Kiểm thử chức năng chính

| Nhóm | Kịch bản | Kết quả kỳ vọng |
|------|----------|-----------------|
|------|----------|-----------------|

|          |                                                      |                                                                                       |
|----------|------------------------------------------------------|---------------------------------------------------------------------------------------|
| Auth     | Đăng ký, đăng nhập, refresh, logout, quên mật khẩu   | Đúng quyền, token hoạt động, mã reset hết hạn và không lộ thông tin email.            |
| Practice | Bắt đầu phiên, nói/gõ, nhận transcript, nghe TTS     | Kết nối LiveKit thành công, không duplicate tin nhắn, agent không chen ngang quá sớm. |
| Widget   | Nói 'cho tôi làm bài tập', 'luyện nghe', 'luyện nói' | Trắc nghiệm mặc định; nghe/nói chỉ hiện khi intent rõ.                                |
| Quiz     | Làm quiz text/listening/speaking và nộp bài          | Chấm điểm, lưu attempt, hiển thị giải thích và AI review.                             |
| Level-up | Tạo bài thi nâng cấp                                 | Bài có text, listening và speaking; pass theo ngưỡng nội bộ.                          |
| Billing  | Hết lượt chat/quiz và tạo payment QR                 | Trả 402/paywall, tạo payment request, admin duyệt mở gói.                             |
| Admin    | Sửa/xóa user, sửa/xóa quiz, import nguồn/PDF         | Chỉ admin truy cập được, dữ liệu cập nhật đúng.                                       |

## 5.2. Đánh giá mức độ hoàn thiện MVP

Kết luận kỹ thuật

- MVP đã đủ để demo hành trình end-to-end: landing → auth → Gia sư AI → widget quiz → kho quiz → review → billing/admin.
- Core AI đã có prompt, LangGraph, STT/TTS, VAD, memory và quiz remediation; chưa phải fine-tune model riêng.
- Các hạn chế nên nói thẳng: chất lượng CEFR là heuristic/prompt + dữ liệu quiz, thanh toán QR còn bán thủ công, phát âm chưa chuẩn âm vị.

## 5.3. Plan demo 30 phút

| Thời lượng | Nội dung demo                                 | Ý cần nói                                                          |
|------------|-----------------------------------------------|--------------------------------------------------------------------|
| 0-3 phút   | Mở landing page, nêu bài toán                 | Khác ChatGPT: có vòng học, quiz, review, admin, quota.             |
| 3-6 phút   | Đăng ký/đăng nhập, giải thích role            | JWT access + refresh cookie, learner/admin tách biệt.              |
| 6-12 phút  | Vào Gia sư AI, nói/gõ 2-3 lượt                | LiveKit WebRTC, STT/TTS, transcript, VAD, system prompt CEFR.      |
| 12-16 phút | Yêu cầu 'cho tôi làm bài tập'                 | Widget quiz trắc nghiệm xuất hiện ngay trong chat từ ngân hàng DB. |
| 16-20 phút | Mở kho quiz, làm một câu nghe/nói/trắc nghiệm | Quiz theo bộ, level lock, AI review kết quả.                       |
| 20-23 phút | Tạo quiz ôn lỗi hoặc xem review               | Dữ liệu lỗi/attempt biến thành bài luyện cá nhân hóa.              |
| 23-25 phút | Thi nâng cấp                                  | Đánh giá nội bộ CEFR gồm text, listening, speaking.                |

|            |                            |                                                                     |
|------------|----------------------------|---------------------------------------------------------------------|
| 25-27 phút | Billing QR và paypal/quota | Free/Plus/Ultra, hết lượt thì chặn chat/quiz và đề nghị nâng cấp.   |
| 27-30 phút | Admin                      | Quản lý user, đổi CEFR, CRUD quiz, import nguồn/PDF, duyệt payment. |

## 5.4. Các câu hỏi bảo vệ dễ gặp

| Câu hỏi                        | Cách trả lời ngắn gọn                                                                                                                              |
|--------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| System prompt ở đâu?           | backend/app/agents/english_tutor/prompts.py. Prompt chính ENGLISH_TUTOR_PROMPT kiểm soát CEFR, topic, memory, style sửa lỗi và độ dài câu trả lời. |
| WebRTC nằm ở đâu?              | frontend/src/hooks/useLiveKit.ts kết nối room; backend/app/router/token.py cấp token LiveKit; agent tham gia room trong endpoint.py.               |
| TTS/STT dùng gì?               | AgentSession dùng EdgeTTS để phát giọng tutor và WhisperSTT để nhận lời nói. VAD Silero quyết định điểm kết thúc lượt nói.                         |
| Widget chat hoạt động thế nào? | Agent gửi JSON qua LiveKit text stream topic ai-quiz-widget/ai-chat-widget; FE parse payload và render InlineQuizWidget/ChatWidgetCard.            |
| AI có fine-tune không?         | Chưa fine-tune. Hệ thống dùng prompt engineering, structured output, LangGraph, memory và dữ liệu quiz để kiểm soát hành vi theo CEFR.             |
| Quiz lấy từ đâu?               | Có curated seed, import nguồn mở/URL, import PDF sách, admin CRUD và AI generation. Các câu mới được lưu vào quiz_sets/quizzes.                    |
| Thanh toán có tự động không?   | Hiện là QR bán tự động: user tạo payment request, chuyển khoản, admin xác nhận. Có thể nâng cấp payOS sau.                                         |

## KẾT LUẬN

Đồ án đã phát triển EnglishAltutor từ ý tưởng trợ lý luyện nói thành một nền tảng học tiếng Anh có đầy đủ các phần cần thiết cho MVP: giao diện learner, giao diện admin, auth, Gia sư AI realtime, STT/TTS, widget trong chat, kho quiz, AI review, thi nâng cấp, billing QR và quota gói học.

Giá trị cốt lõi của hệ thống nằm ở việc nối các hoạt động học lại với nhau. Người học không chỉ nói chuyện với AI mà còn có dữ liệu lỗi, bài ôn, bài quiz, review và lộ trình. Admin có công cụ để quản lý người dùng, nội dung và payment, giúp sản phẩm có thể vận hành trong bối cảnh demo thực tế.

Hướng phát triển tiếp theo gồm: tích hợp payOS để xác nhận giao dịch tự động, mở rộng ngân hàng đề có kiểm định chất lượng, đánh giá phát âm chi tiết hơn, bổ sung analytics dài hạn và kiểm thử tải cho nhiều phiên voice đồng thời.

## TÀI LIỆU THAM KHẢO

1. Council of Europe, Common European Framework of Reference for Languages: Learning, Teaching, Assessment.

- 2. LiveKit Documentation, Rooms, Agents, WebRTC and text streams.
- 3. FastAPI Documentation, Security and dependency injection.
- 4. LangGraph Documentation, StateGraph and agent orchestration.
- 5. React and Vite Documentation.
- 6. PostgreSQL Documentation.
- 7. Wikibooks English Grammar and Tatoeba Project licensing notes for open language resources.

PHỤ LỤC. VỊ TRÍ CODE QUAN TRỌNG

| Thành phần         | File chính                                                           | Ghi chú                                                      |
|--------------------|----------------------------------------------------------------------|--------------------------------------------------------------|
| System prompt      | backend/app/agents/english_tutor/prompts.py                          | ENGLISH_TUTOR_PROMPT, ERROR_ANALYSIS_PROMPT, ROUTER_PROMPT.  |
| Agent realtime     | backend/app/agents/english_tutor/entrypoint.py                       | LiveKit job, memory, LangGraph, TTS/STT/VAD, widget trigger. |
| LangGraph          | backend/app/agents/english_tutor/graph.py                            | Tạo graph<br>assess/respond/correct/save_memory.             |
| Metrics/CEFR       | backend/app/agents/english_tutor/session_metrics.py                  | Tính session stats, working_level, scorecard.                |
| WebRTC frontend    | frontend/src/hooks/useLiveKit.ts                                     | Kết nối room, transcript, text stream, widget stream.        |
| Inline quiz widget | frontend/src/components/chat/InlineQuizWidget.tsx                    | Render quiz/listening/speaking trong khung chat.             |
| Quiz API           | backend/app/router/quizzes.py                                        | CRUD, import, generate, submit, AI review, level-up.         |
| Auth/JWT           | backend/app/core/auth.py và backend/app/router/auth.py               | JWT, refresh token, role, reset password.                    |
| Billing/quota      | backend/app/core/subscriptions.py và backend/app/router/billing.py   | Free/Plus/Ultra, usage, QR payment.                          |
| Admin UI           | frontend/src/pages/AdminUsers.tsx, QuizStudio.tsx, AdminPayments.tsx | Quản trị user, quiz, payment.                                |